NUMB11: Numerical Linear Algebra: Assignment 2

Arpita Bhattacharya
20010823-7348

September 16, 2024

1. **Task 1 solution:**
In this Task, we are asked to apply the Gram-Schmidt orthogonalization to a given $m \times n$ matrix A with $m \geq n$.

Using the concept of QR-factorization, we want to write matrix $A = QR$ where Q is a $m \times n$ matrix of orthonormal columns that span the successive subspaces spanned by the columns of A. R is a $n \times n$ upper-triangular matrix.

$$\begin{bmatrix} \mathbf{a_1} & \mathbf{a_2} & \cdots & \mathbf{a_n} \end{bmatrix} = \begin{bmatrix} \mathbf{q_1} & \mathbf{q_2} & \cdots & \mathbf{q_n} \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & 0 & & r_{nn} \end{bmatrix} \tag{1}$$

a_1 and q_1 can be written as the linear combination of each other:

$$\begin{aligned} a_1 &= r_{11}q_1, \\ a_2 &= r_{12}q_1 + r_{22}q_2, \\ a_3 &= r_{13}q_1 + r_{23}q_2 + r_{33}q_3, \\ &\vdots \\ a_n &= r_{1n}q_1 + r_{2n}q_2 + \cdots + r_{nn}q_n, \end{aligned} \tag{2}$$

These equations is used to compute the reduced QR-factorizations. That is, given $a_1, a_2, ...$, we can construct vectors $q_1, q_2, ...$ and entries r_{ij} by the Gram-Schmidt orthogonalization method.

In this method, at the j-th step, we will find a unit vector q_j that is orthogonal to $q_1, ...q_{j-1}$. In my python code 1.1, I have done this under the method `gramschmidt()`.

In this, I first run an outer loop where every column vector a_j is assigned to a vector v_j. So, this is the current vector that we will orthogonalize against the previous vectors. Then I run a second inner loop for this orthogonalization. This loop iterates over all previously computed orthonormal vectors q_i for $i < j$, to make v_j orthogonal to them. We will subtract the projection of v_j onto q_i to remove its component in that direction, making v_j orthogonal to q_i. Or,

$$v_j = a_j - (q_1^T a_j)q_1 - (q_2^T a_j)q_2 - \cdots - (q_{j-1}^T a_j)q_{j-1} \tag{3}$$

Now, if we rewrite the equations (2) as,

$$\begin{aligned} q_1 &= \frac{a_1}{r_{11}}, \\ q_1 &= \frac{a_1 - r_{12}q_1}{r_{22}}, \\ q_1 &= \frac{a_1 - r_{13}q_1 - r_{23}q_2}{r_{33}}, \\ &\vdots \\ q_1 &= \frac{a_n - \sum_{i=1}^{n-1} r_{in}q_i}{r_{nn}}, \end{aligned} \tag{4}$$

Comparing this with equation (3), we have,

$$r_{ij} = q_i^T a_j \quad (i \neq j) \tag{5}$$

The coefficients r_{jj} in the denominators are chosen for normalization:

$$|r_{jj}| = ||a_j - \sum_{i=1}^{j-1} r_{ij}q_i||_2 \tag{6}$$

We then finally normalize the vector v_j by dividing it by $||v_j||_2$. These vectors form the matrix Q and are the n orthogonal basis vectors of range(A).

2. **Task 2 solution:**
In this Task, we are asked to test the code in Task 1 by choosing some random matrices. That is, to measuring the orthogonality of the matrix by a few different criteria. We are also required to test it with increasing dimensions of matrix A.

Let us first go through each criteria and and write what outcomes we expect to occur:

- Is the 2-norm one?
  - For an orthonormal matrix, each column vector should have a 2-norm equal to 1.
- How big is the deviation of Q^TQ from the identity matrix? Measure this in 2-norm.
  - For an orthonormal matrix, Q^TQ should equal the identity matrix. If Q is perfectly orthonormal, $Q^TQ - I$ should be close to zero. The 2-norm of this difference matrix gives us a measure of deviation.
- Compute the eigenvalues of Q^TQ. What do you expect they should be for an orthogonal matrix?
  - For an orthonormal matrix, all the eigenvalues of Q^TQ should be 1.
- Compute the determinant.
  - For an orthonormal matrix, the determinant should either be 1 or -1.

To measure the orthogonality based on the above criteria on some random matrices, we will use the code as written in 1.2. The output is also given there. Note that to find the determinant, Q needs to be a square matrix. Thus, I have chosen 3×3 matrices.

Now, in the code in 1.2, the method `checkorthogonality()` measures the orthogonality by checking each of the given criteria. And as shown in the output 1.2, we have got what we expected, as listed above, the outcome to be.

Next, we will test the orthogonality with matrices A with increasing dimensions We will test this with the example already given in the task, that is, for $m = n + 2$ and $n = 1, 10, 100, 1000, 10000$, and so on. This is also written in 1.2.

Result:
I noticed that for dimension $m = n + 2$ as n increases in each test from 1 to $10, 100, 1000$, and so on, it gave the expected outcomes for all the criteria (except determinant), hence measuring the orthogonality.

But if we take $m = n$ instead of $m = n+2$, we are getting results for the determinant too. For smaller dimensions, the determinant is either 1 or -1, but as the dimensions increases, the determinant is extremely close to 1 or -1, but not exactly 1 or -1.

The main observation was that the last output it showed was for $n = 1000$, and then for $n = 10000$, the execution showed to be 'busy' and basically got stuck running the code to show any output. I needed to externally stop the program from running. This is showing us that as the dimensions get bigger, the slower and more unstable the execution becomes for the code.

3. **Task 3 solution:**
In this Task, we will first compute the orthogonal basis of range(A) by using scipy.linalg's command `qr`. Doing this means that we can remove the entire `gramschmidt()` method and instead replace it with one line of code.

Now, I have re-written the code from Task 2 but with mentioned replacement to command `qr` in 1.3. We observe the following by reiterating Task 1 and 2 here:

- For the orthonormal matrix, each column vector gave a 2-norm equal to 1.
- For the perfectly orthonormal matrix Q, $(Q^TQ - I)$ was close to zero. The 2-norm of this difference matrix gave us the measure of deviation.
- For the orthonormal matrix, all the eigenvalues of Q^TQ was equal to 1.
- For the orthonormal matrix, the determinant was very close to being either 1 or -1.
- As the dimensions increased, the execution was smoother, faster and more efficient. Unlike previously, it didn't lead to the code getting stuck.

Qualitative difference:
The QR decomposition using scipy.linalg.qr is more numerically stable than the manually implemented Gram-Schmidt orthogonalization, especially for matrices with larger dimensions. The command `qr` provides a built-in way to get the orthogonal basis of the range(A) more efficiently.

Gram-Schmidt is unstable:
The algorithm used in the Tasks 1 and 2 for the Gram-Schmidt orthogonalization, mathematically offers a simple route to understanding and proving various properties of QR-factorizations. But, numerically, it turns out to be less efficient and unstable because of rounding errors on a computer.

4. **Task 4 solution:**
This task is based on the Householder transformations. Here, we will write a python program (as a method of previously used class `Orthagonalization`) performing this transformation. The code is 1.4

In the code we can see that in the method `householder_transf()`, we first initialize A, Q and its dimensions. We then get into a for loop. This loop goes over each column of the matrix A. The algorithm processes one column at a time to construct the upper triangular matrix R and the orthogonal matrix Q.

Then, we first extract the vector x from the k-th column of A starting from the k-th row to the end of the column. This is the portion of the column that will be transformed in the current iteration to zero out elements below the diagonal.

Next we compute the householder vector. We start by computing the 2-norm (Euclidean norm) of the vector x. This norm is used to calculate the scaling factor for the Householder vector. The 'sign' determines the sign of the first element of x to ensure numerical stability and to prevent cancellation errors during reflection.

We then create the householder vector v_k using the formula:

$$v_k = \text{sign}(x_1) * ||x||_2 * e_1 + x \tag{7}$$

`np.eye(len(x))[:, 0]` creates a standard basis vector e_1 of the same length as x. `sign * norm_x * np.eye(len(x))[:, 0]` scales this basis vector, and adding it to x forms the Householder vector.

Next we normalize the vector v_k to ensure it has unit length. This is essential for forming the reflection.

We then project the submatrix of A onto the Householder vector v_k. The outer product then forms the Householder matrix used for the reflection. And then we subtract the reflected component to zero out elements in the submatrix of A. This process progressively transforms A into an upper triangular matrix R.

We finally accumulate the Householder transformations to build the orthogonal matrix Q. We do this by projecting the columns of Q onto the Householder vector v_k, forming the outer product to construct the reflection matrix and applying the reflection to the relevant columns of Q.

At the end of the loop, the matrix A has been transformed into an upper triangular matrix, which is now the matrix R.

Basically, each iteration of the loop constructs a Householder vector to zero out elements below the diagonal in the current column, transforming the matrix into an upper triangular form (R). The product of all the Householder transformations is accumulated to form the orthogonal matrix Q.

Now let's test our code:

- Checking if Q is indeed orthogonal.
  To check if the matrix Q is orthogonal, we need to verify that the product of Q and its transpose Q^T is the identity matrix. We can do this by the second code 1.4.
- Checking if A = QR
  To check this, we only have to find the product of Q and R that we found through the code and compare with the given matrix A. This can be done using the third code under 1.4.
- Comparing the result to Python's scipy.linalg.qr.
  The fourth code in 1.4 is using Python's scipy.linalg.qr. It is observed that this is also giving the same results as the Householder transformations.

NOTE: I will resubmit the assignment with the solution of Tasks 5 and 6 as soon as possible. Please do provide feedback for Task 1- 4 until then. Thank you! Sincerely, Arpita.

5. **Task 5 solution:**
   In Task 4, we have considered reflections to orthogonalize a matrix which led to the Householder transformations to perform a QR-factorization. In this task, we will use an alternative, that is, we will use rotations for the same purpose. This is the *Givens Rotations*.

   (a) How is rotation in $\mathbb{R}^n$ described?

6. **Task 6 solution:**

1 Appendix

1.1 Task 1

```python
import numpy as np

class Orthogonalization:
    def __init__(self, A):

        self.A = np.array(A)
        self.m, self.n = self.A.shape

        if self.m < self.n:
            raise ValueError("The number of rows m is less than the number of
                                                    columns n.")

    def gramschmidt(self):

        Q = np.zeros((self.m, self.n))  # Matrix of the orthonormal vectors
        R = np.zeros((self.n, self.n))  # Upper triangular matrix

        for j in range(self.n):
            v_j = self.A[:, j]  # Start with the j-th column of A

            # Subtract the projection onto the previous q_i vectors
            for i in range(j):
                q_i = Q[:, i]
                R[i, j] = np.dot(q_i, v_j)  # r_ij = q_i^T * a_j
                v_j = v_j - R[i, j] * q_i  # v_j = v_j - r_ij * q_i

            # Compute r_jj = ||v_j||_2 (2-norm of v_j)
            R[j, j] = np.linalg.norm(v_j, 2)

            # Normalize v_j to get q_j
            Q[:, j] = v_j / R[j, j]

        return Q, R
```

Add the following code to the above one as an example to find the orthogonal basis.

```python
if __name__ == "__main__":
    A = [[1, 1, 0],
         [1, 0, 1],
         [0, 1, 1]]

    # Instantiate the class
    orth = Orthogonalization(A)

    # Perform Classical Gram-Schmidt orthogonalization
    Q, R = orth.gramschmidt()

    # Print the orthonormal basis vectors Q and the upper triangular matrix R
    print("Orthogonal basis vectors Q:")
    print(Q)
    print("\nUpper triangular matrix R:")
    print(R)
```

1.2 Task 2

Checking criteria for orthogonality:

```python
import numpy as np

class Orthogonalization:
    def __init__(self, A):

        self.A = np.array(A)
        self.m, self.n = self.A.shape

        if self.m < self.n:
            raise ValueError("The number of rows m is less than the number of
                                                        columns n.")

    def gramschmidt(self):

        Q = np.zeros((self.m, self.n))  # Matrix of the orthonormal vectors
        R = np.zeros((self.n, self.n))  # Upper triangular matrix

        for j in range(self.n):
            v_j = self.A[:, j]  # Start with the j-th column of A

            # Subtract the projection onto the previous q_i vectors
            for i in range(j):
                q_i = Q[:, i]
                R[i, j] = np.dot(q_i, v_j)  # r_ij = q_i^T * a_j
                v_j = v_j - R[i, j] * q_i  # v_j = v_j - r_ij * q_i

            # Compute r_jj = ||v_j||_2 (2-norm of v_j)
            R[j, j] = np.linalg.norm(v_j, 2)

            # Normalize v_j to get q_j
            Q[:, j] = v_j / R[j, j]

        return Q, R

def checkorthogonality(Q):
    # Check if the 2-norm of each column of Q is 1
    column_norms = np.linalg.norm(Q, axis=0)
    print("2-norm of each column of Q:", column_norms)

    # Deviation of Q^T Q from the identity matrix (in 2-norm)
    I = np.eye(Q.shape[1])
    Q_T_Q = np.dot(Q.T, Q)
    deviation_norm = np.linalg.norm(Q_T_Q - I, ord=2)
    is_close_to_identity = np.allclose(Q_T_Q, I, atol=1e-8)
    print("Deviation of Q^T Q from identity (2-norm):", deviation_norm)
    print("Q^T Q is close to identity:", is_close_to_identity)

    # Eigenvalues of Q^T Q
    eigenvalues = np.linalg.eigvals(Q_T_Q)
    print("Eigenvalues of Q^T Q:", eigenvalues)

    # Determinant of Q (only if Q is square)
    if Q.shape[0] == Q.shape[1]:
        determinant = np.linalg.det(Q)
        print("Determinant of Q:", determinant)
```

```
    else:
        print("Determinant is not defined for non-square Q matrix.")


if __name__ == "__main__":
    # Create a random 3x3 matrix (m >= n)
    A = np.random.rand(3, 3)

    # Perform Gram-Schmidt orthogonalization
    orthogonalizer = Orthogonalization(A)
    Q, R = orthogonalizer.gramschmidt()

    # Check the orthogonality properties
    checkorthogonality(Q)
```

Output:

```
2-norm of each column of Q: [1. 1. 1.]
Deviation of Q^T Q from identity (2-norm): 4.279244400882571e-16
Q^T Q is close to identity: True
Eigenvalues of Q^T Q: [1. 1. 1.]
Determinant of Q: 1.0
```

Code for increasing dimensions of A:

```
import numpy as np

class Orthogonalization:
    def __init__(self, A):

        self.A = np.array(A)
        self.m, self.n = self.A.shape

        if self.m < self.n:
            raise ValueError("The number of rows m is less than the number of
                                                    columns n.")

    def gramschmidt(self):

        Q = np.zeros((self.m, self.n))  # Matrix of the orthonormal vectors
        R = np.zeros((self.n, self.n))  # Upper triangular matrix

        for j in range(self.n):
            v_j = self.A[:, j]  # Start with the j-th column of A

            # Subtract the projection onto the previous q_i vectors
            for i in range(j):
                q_i = Q[:, i]
                R[i, j] = np.dot(q_i, v_j)  # r_ij = q_i^T * a_j
                v_j = v_j - R[i, j] * q_i  # v_j = v_j - r_ij * q_i

            # Compute r_jj = ||v_j||_2 (2-norm of v_j)
            R[j, j] = np.linalg.norm(v_j, 2)

            # Normalize v_j to get q_j
            Q[:, j] = v_j / R[j, j]
```

```
        return Q, R

def checkorthogonality(Q):
    # Check if the 2-norm of each column of Q is 1
    column_norms = np.linalg.norm(Q, axis=0)
    print("2-norm of each column of Q:", column_norms)

    # Deviation of Q^T Q from the identity matrix (in 2-norm)
    I = np.eye(Q.shape[1])
    Q_T_Q = np.dot(Q.T, Q)
    deviation_norm = np.linalg.norm(Q_T_Q - I, ord=2)
    is_close_to_identity = np.allclose(Q_T_Q, I, atol=1e-8)
    print("Deviation of Q^T Q from identity (2-norm):", deviation_norm)
    print("Q^T Q is close to identity:", is_close_to_identity)

    # Eigenvalues of Q^T Q
    eigenvalues = np.linalg.eigvals(Q_T_Q)
    print("Eigenvalues of Q^T Q:", eigenvalues)

    # Determinant of Q (only if Q is square)
    if Q.shape[0] == Q.shape[1]:
        determinant = np.linalg.det(Q)
        print("Determinant of Q:", determinant)
    else:
        print("Determinant is not defined for non-square Q matrix.")

# Testing with matrices of increasing dimensions
dimensions = [1, 10, 100, 1000, 10000]
for n in dimensions:
    m = n + 2
    print(f"\nTesting with matrix of dimensions {m} x {n}")

    # Create a random m x n matrix
    A = np.random.rand(m, n)

    # Perform Gram-Schmidt orthogonalization
    orthogonalizer = Orthogonalization(A)
    Q, R = orthogonalizer.gramschmidt()

    # Check orthogonality properties
    checkorthogonality(Q)
```

1.3 Task 3

Computes orthogonal basis (Q), checks orthogonality using list of criteria from Task 2 and does so with increasing dimensions of matrix A:

```
import numpy as np
from scipy.linalg import qr

class Orthogonalization:
    def __init__(self, A):

        self.A = np.array(A)
        self.m, self.n = self.A.shape
```

```
        if self.m < self.n:
            raise ValueError("The number of rows m is less than the number of
                                                    columns n.")

def checkorthogonality(Q):
    # Check if the 2-norm of each column of Q is 1
    column_norms = np.linalg.norm(Q, axis=0)
    print("2-norm of each column of Q:", column_norms)

    # Deviation of Q^T Q from the identity matrix (in 2-norm)
    I = np.eye(Q.shape[1])
    Q_T_Q = np.dot(Q.T, Q)
    deviation_norm = np.linalg.norm(Q_T_Q - I, ord=2)
    is_close_to_identity = np.allclose(Q_T_Q, I, atol=1e-8)
    print("Deviation of Q^T Q from identity (2-norm):", deviation_norm)
    print("Q^T Q is close to identity:", is_close_to_identity)

    # Eigenvalues of Q^T Q
    eigenvalues = np.linalg.eigvals(Q_T_Q)
    print("Eigenvalues of Q^T Q:", eigenvalues)

    # Determinant of Q (only if Q is square)
    if Q.shape[0] == Q.shape[1]:
        determinant = np.linalg.det(Q)
        print("Determinant of Q:", determinant)
    else:
        print("Determinant is not defined for non-square Q matrix.")

# Testing with matrices of increasing dimensions
dimensions = [1, 10, 100, 1000, 10000]
for n in dimensions:
    m = n + 2
    print(f"\nTesting with matrix of dimensions {m} x {n}")

    # Create a random m x n matrix
    A = np.random.rand(m, n)

    # Perform Gram-Schmidt orthogonalization
    Q, R = qr(A, mode='full')

    # Check orthogonality properties
    checkorthogonality(Q)
```

1.4 Task 4

```
import numpy as np

class Orthogonalisation:
    def __init__(self, A):
        # Ensure A is an m x n numpy array
        self.A = np.array(A, dtype=float)
        self.m, self.n = self.A.shape
        if self.m < self.n:
            raise ValueError("The matrix A must have more rows than columns (m >= n)
                                                    ")

    def householder_transf(self):
```

```
        A = self.A.copy()
        m, n = self.m, self.n
        Q = np.eye(m)

        for k in range(n):
            # Step 1: Extract the vector x (from A[k:m, k])
            x = A[k:m, k]

            # Step 2: Compute the Householder vector v_k
            norm_x = np.linalg.norm(x)
            sign = -1 if x[0] < 0 else 1
            v_k = x + sign * norm_x * np.eye(len(x))[:, 0]  # v_k = sign(x_1) * ||x|
                                                            |_2 * e_1 + x

            # Normalize v_k
            v_k = v_k / np.linalg.norm(v_k)

            # Step 3: Update A using the Householder transformation
            A[k:m, k:n] -= 2 * np.outer(v_k, np.dot(v_k, A[k:m, k:n]))

            # Update Q (accumulating Householder transformations)
            Q[:, k:m] -= 2 * np.outer(Q[:, k:m] @ v_k, v_k)

        # R is the modified A
        R = A
        return Q, R

# Example usage:
m=3
n=3
A = np.random.rand(m, n)
ortho = Orthogonalisation(A)
Q, R = ortho.householder_transf()
print("Q:\n", Q)
print("R:\n", R)
```

Add the following code to the above code to check if Q is orthogonal:

```
# Check orthogonality of Q
identity_check = np.allclose(np.dot(Q.T, Q), np.eye(Q.shape[0]))
print("Is Q orthogonal?", identity_check)

# Output Q^T * Q
print("Q^T * Q:")
print(np.dot(Q.T, Q))
```

Add the following code to verify that $A = QR$:

```
# Check if A is approximately equal to Q @ R
A_reconstructed = np.dot(Q, R)
is_equal = np.allclose(A_reconstructed, A)
print("\nIs A equal to QR?", is_equal)

# Output the original matrix A and the reconstructed QR product
print("\nOriginal matrix A:")
print(A)
print("\nReconstructed matrix QR:")
print(A_reconstructed)
```

Output:

```
Q:
 [[-0.14009343  0.73211979  0.66661416]
 [-0.9748454  -0.21985781  0.03659211]
 [-0.17335014  0.64471944 -0.74450425]]
R:
 [[-9.11457707e-01 -4.73755528e-01 -5.68797650e-01]
 [ 1.11022302e-16  9.44524239e-01  2.52738055e-01]
 [ 2.77555756e-17 -1.11022302e-16 -1.35146763e-01]]
Is Q orthogonal? True
Q^T * Q:
[[1.00000000e+00 1.73966038e-16 1.42904760e-16]
 [1.73966038e-16 1.00000000e+00 2.46477500e-16]
 [1.42904760e-16 2.46477500e-16 1.00000000e+00]]

Is A equal to QR? True

Original matrix A:
[[0.12768924 0.75787493 0.1746286 ]
 [0.88853036 0.25417736 0.49397803]
 [0.15800132 0.69107872 0.36216363]]

Reconstructed matrix QR:
[[0.12768924 0.75787493 0.1746286 ]
 [0.88853036 0.25417736 0.49397803]
 [0.15800132 0.69107872 0.36216363]]
```

Using `qr` command instead of the householder method:

```python
import numpy as np
from scipy.linalg import qr

class Orthogonalisation:
    def __init__(self, A):
        # Ensure A is an m x n numpy array
        self.A = np.array(A, dtype=float)
        self.m, self.n = self.A.shape
        if self.m < self.n:
            raise ValueError("The matrix A must have more rows than columns (m >= n)
                                                                    ")


# Example usage:
m=3
n=3
A = np.random.rand(m, n)
Q, R = qr(A)
print("Q:\n", Q)
print("R:\n", R)

# Check orthogonality of Q
identity_check = np.allclose(np.dot(Q.T, Q), np.eye(Q.shape[0]))
print("Is Q orthogonal?", identity_check)

# Output Q^T * Q
print("Q^T * Q:")
```

```
print(np.dot(Q.T, Q))

# Check if A is approximately equal to Q @ R
A_reconstructed = np.dot(Q, R)
is_equal = np.allclose(A_reconstructed, A)
print("\nIs A equal to QR?", is_equal)

# Output the original matrix A and the reconstructed QR product
print("\nOriginal matrix A:")
print(A)
print("\nReconstructed matrix QR:")
print(A_reconstructed)
```